

Number: P3449R1

Date: 2024-12-16

Audience: LWG

Reply-to: [Hana.Dusíková](#)

Table of contents

- [Changes](#)
- [Quote](#)
- [Example](#)
- [Implementation experience](#)
- [Impact of existing code](#)
- [Intention for wording changes](#)
- [Proposed wording changes](#)
 - [Feature test macro](#)

constexpr generator

This paper is proposing making `std::generator` `constexpr` and it depends on [P3367: constexpr coroutines](#). I was told to put it into a separate paper, so here it is. I don't want this only coroutine type to lag behind language as before.

Changes

- [R0](#) → [R1](#): Removal (from previous paper) of `constexpr` support for alternative allocators (these functions are elided during constant evaluation of coroutines).

Quote

Well, you just told me coroutines are the best way to solve some problems, so wouldn't I also want to use the Best Way at compile time? (quote from Jason Turner, co-author of "*constexpr all the things*" talk)

Example

When this paper is merged into the standard, users will be able to use `std::generator`-like coroutines to generate or calculate data.

```
template <typename T> constexpr auto fib() {
    T a = 0;
    T b = 1;
    co_yield a;
    do {
        co_yield b;
        auto tmp = b;
        b += a;
        a = tmp;
    } while (true);
}

template <typename T, size_t N> constexpr auto fib() {
    auto res = std::array<T, N>{};
    std::ranges::copy(fib<T>() | std::views::take(N), res.begin());
    return res;
}
```